



AAAI CONFERENCE
AAAI Conference on
Artificial Intelligence

EoH-S: Evolution of Heuristic Set Using LLMs for Automated Heuristic Design

基于 LLM 的启发式集合在自动启发式设计中的进化

AAAI `2026

Fei Liu^{1*}, Yilu Liu^{1*}, Qingfu Zhang^{1†}, Xialiang Tong²
, Mingxuan Yuan²

¹City University of Hong Kong

²Huawei Noah' s Ark Lab



香港城市大學
City University of Hong Kong

汇报人：范传华



NOAH'S ARK LAB

From Big Data To Deep Knowledge™

作者介绍

共一



Dr. Fei LIU, 柳斐

研究方向:

- 自动化算法设计
- 组合优化
- 进化计算

共一



Yilu LIU, 劉一鷺

研究方向:

- Few For Many
- 多目标优化
- 计算社会科学

通讯



Prof. Qingfu ZHANG
张青富

研究方向:

- 进化计算
- 多目标优化
- 计算智能

标题: Hierarchical Representations for Cross-task Automated Heuristic Design using LLMs

发表会议: ICML

标题: Deep Uncertainty-aware Tracking for Maneuvering Targets

发表期刊: IEEE Transactions on Aerospace and Electronic Systems

标题: LLM-Enabled Automated Algorithm Design for Multiuser Fluid Antenna Communications

发表期刊: IEEE Transactions on Wireless Communications

标题: Fast Nodal Hessian Computation for Peridynamic Fracture Simulation

发表期刊: Computer Graphics Forum, e70503

标题: A Permutation-Invariant and Variable-Dimension Data-Driven Evolutionary Algorithm for Indoor Antenna Layout Optimization

发表期刊: IEEE Transactions on Systems, Man, and Cybernetics: Systems



香港城市大學
City University of Hong Kong

提纲

1

研究背景

3

系统设计

4

实验评估

5

小结

提纲

1

研究背景

3

系统设计

4

实验评估

5

小结

研究背景

什么是启发式算法？

启发式算法是一类利用经验、规则或直觉，快速找到一个“比较好”的可行解的方法。不追求把所有可能性都算完，而是利用聪明的规则，快速找到一个足够好的答案。

特点：

计算速度较快。

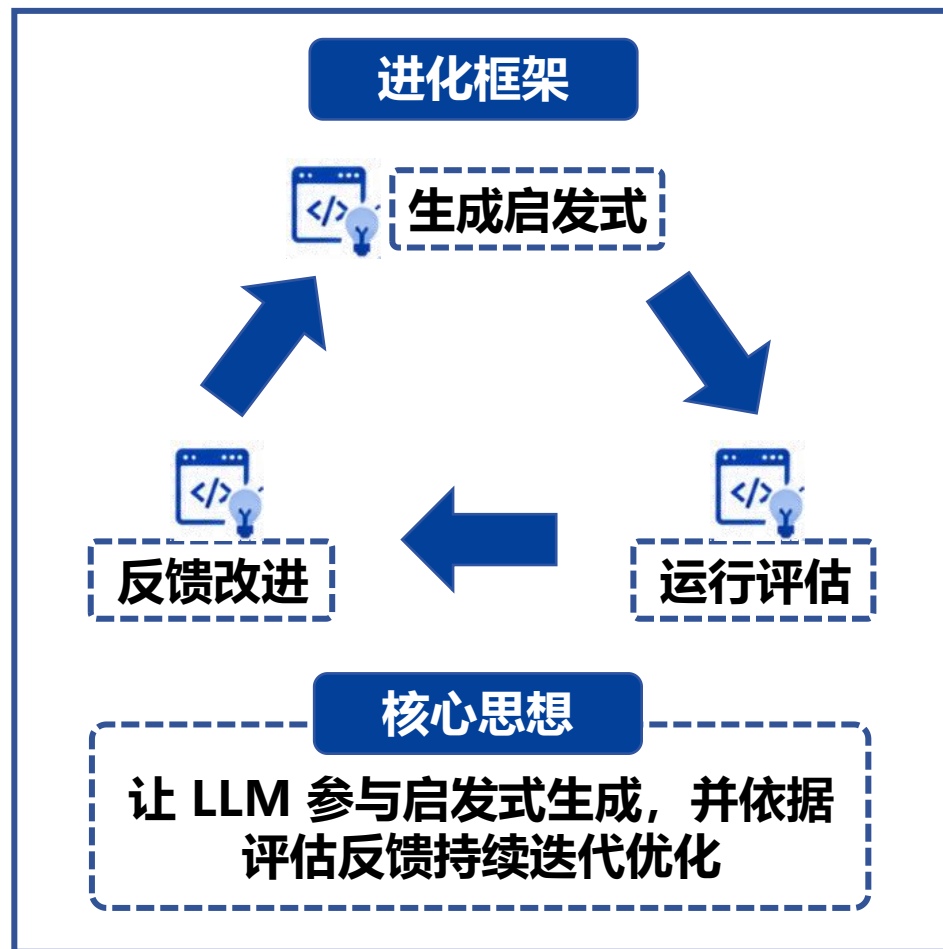
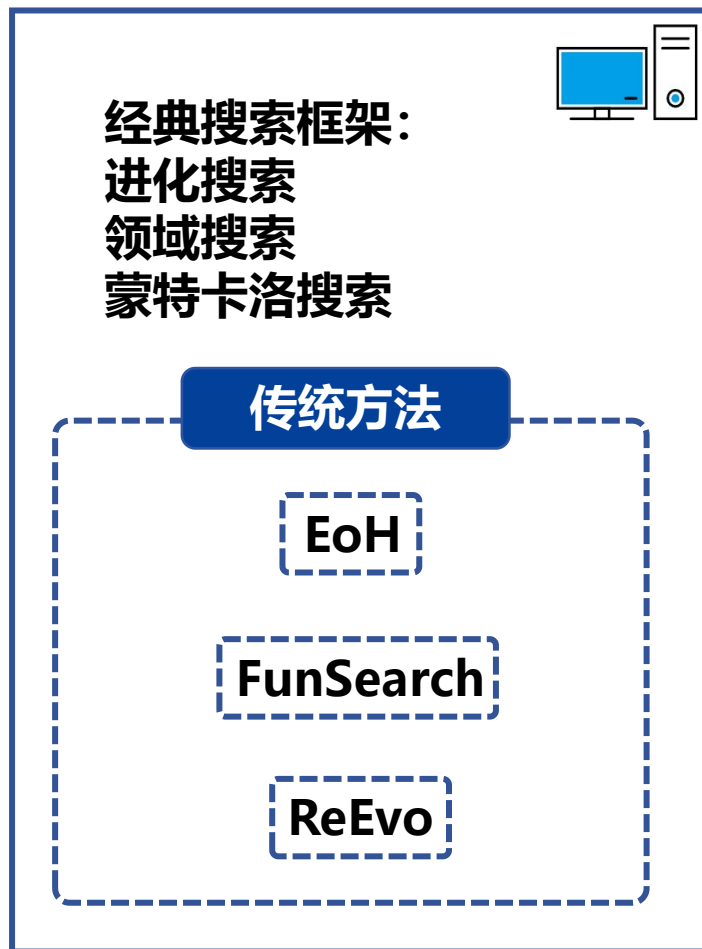
不需要遍历所有可能的方案；

能解决规模较大的复杂问题；

不保证得到全局最优解，但通常能在较短时间内得到质量不错的解。

研究背景

自动设计启发式算法



研究背景

什么是进化搜索、邻域搜索、蒙特卡洛搜索？

进化搜索

像生物进化一样筛选算法
维护一组候选算法，称为种群

流程

生成多个启发式

评估算法性能

修改、组合或变异优秀算法

生成新一代算法

同时培养很多候选算法，让表现好的算法留下来并继续“繁殖和变异”

邻域搜索

在当前算法附近不断修改

流程

从当前解出发，每次进行小范围修改，产生差异较小的“邻居解”，然后选择更好的邻居继续搜索

当前解
A→B→C→D→A
成本：22

更优，移动到这里

邻居解1
A→C→B→D→A
成本：24

邻居解2
A→B→D→C→A
成本：20

先找到一个可用算法，然后每次做一点小修改，逐渐把它改得更好

蒙特卡洛搜索

建立搜索树，重点探索有潜力的方向

步骤

- 1、选择：根据节点价值估计，从根节点选一条路径
- 2、扩展：在叶节点上扩展一个新节点
- 3、评估：测试候选方案的性能
- 4、回传：把评估结果反向回传并更新路径上所有节点的价值估计

初始状态

考虑距离

考虑时间

考虑负载

之前测试表现较好，优先搜索

最早截止

能耗优先

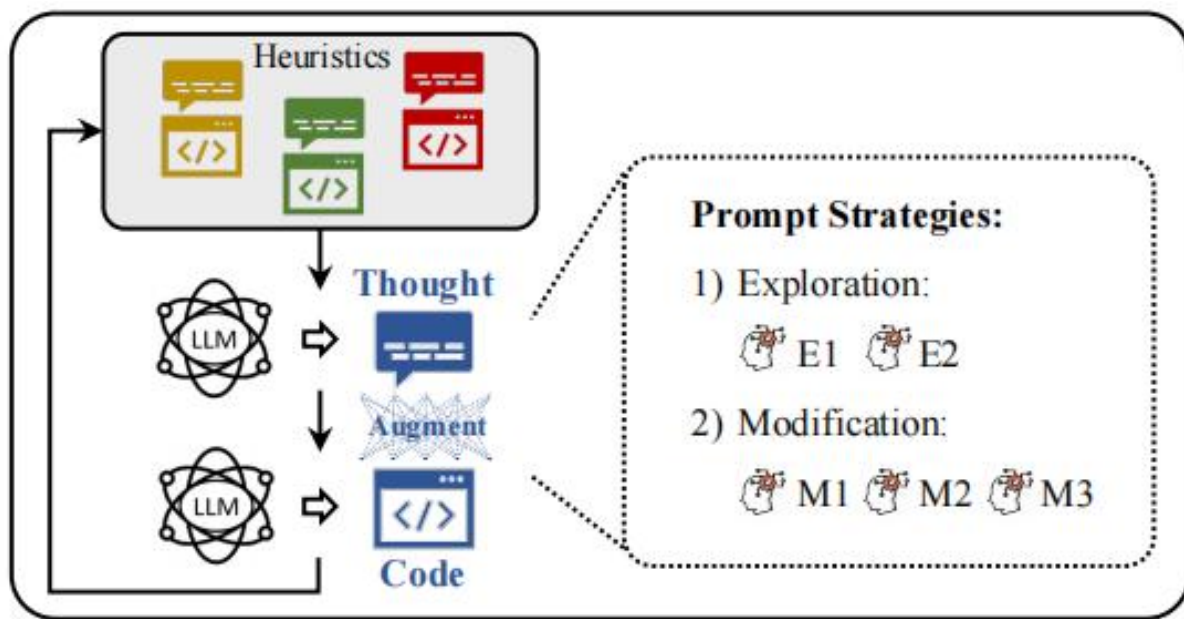
探索新方向

把所有可能的设计过程组织成一棵树，一边尝试新方向，一边重点深入表现好的方向

研究背景 - [ICML `24] EoH

Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model

最终得到一个性能最好的启发式算法



启发式 thought、code、fitness

核心思想 进化算法负责搜索和筛选，LLM 负责生成和修改启发式

EoH

EoH首先维护一个由多个候选启发式组成的种群，每个启发式同时包含 thought、code、fitness (性能分数)

在每一轮进化中，系统从当前种群中选择父代，并通过E1、E2两种探索策略产生新的设计方向，或通过M1、M2、M3三种修改策略对已有启发式进行结构改进、参数调整和冗余精简，随后由 LLM 生成新的 thought、code。

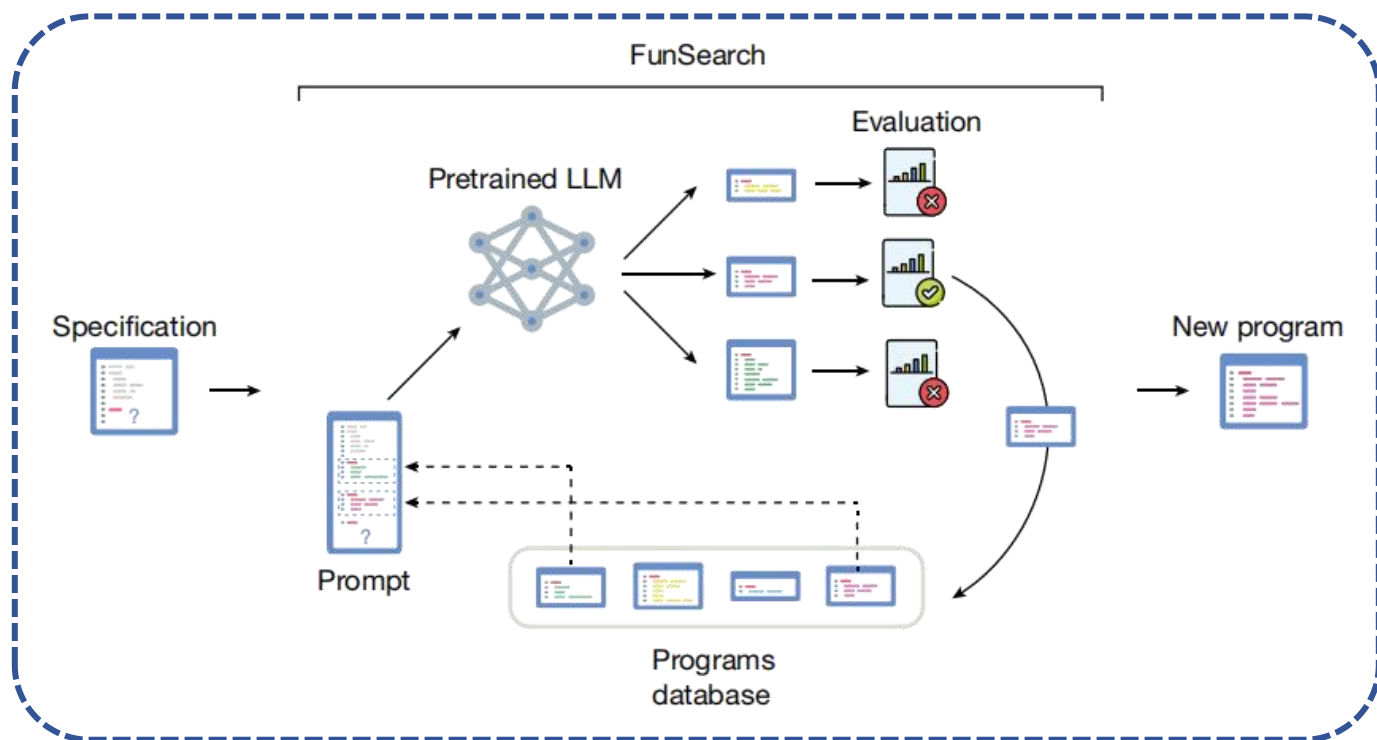
新代码经过合法性检查后，会在一组训练实例上实际运行并计算 fitness，再与原有启发式共同参与筛选，保留性能最好的个体形成下一代种群。

通过不断重复“选择父代—生成 thought 与 code—运行评估—筛选更新”的过程，EoH 最终从逐步优化的种群中得到一个性能最好的启发式算法。

研究背景 - [Nature `24] FunSearch

Mathematical discoveries from program search with large language models

最终得到一个性能最好的启发式算法



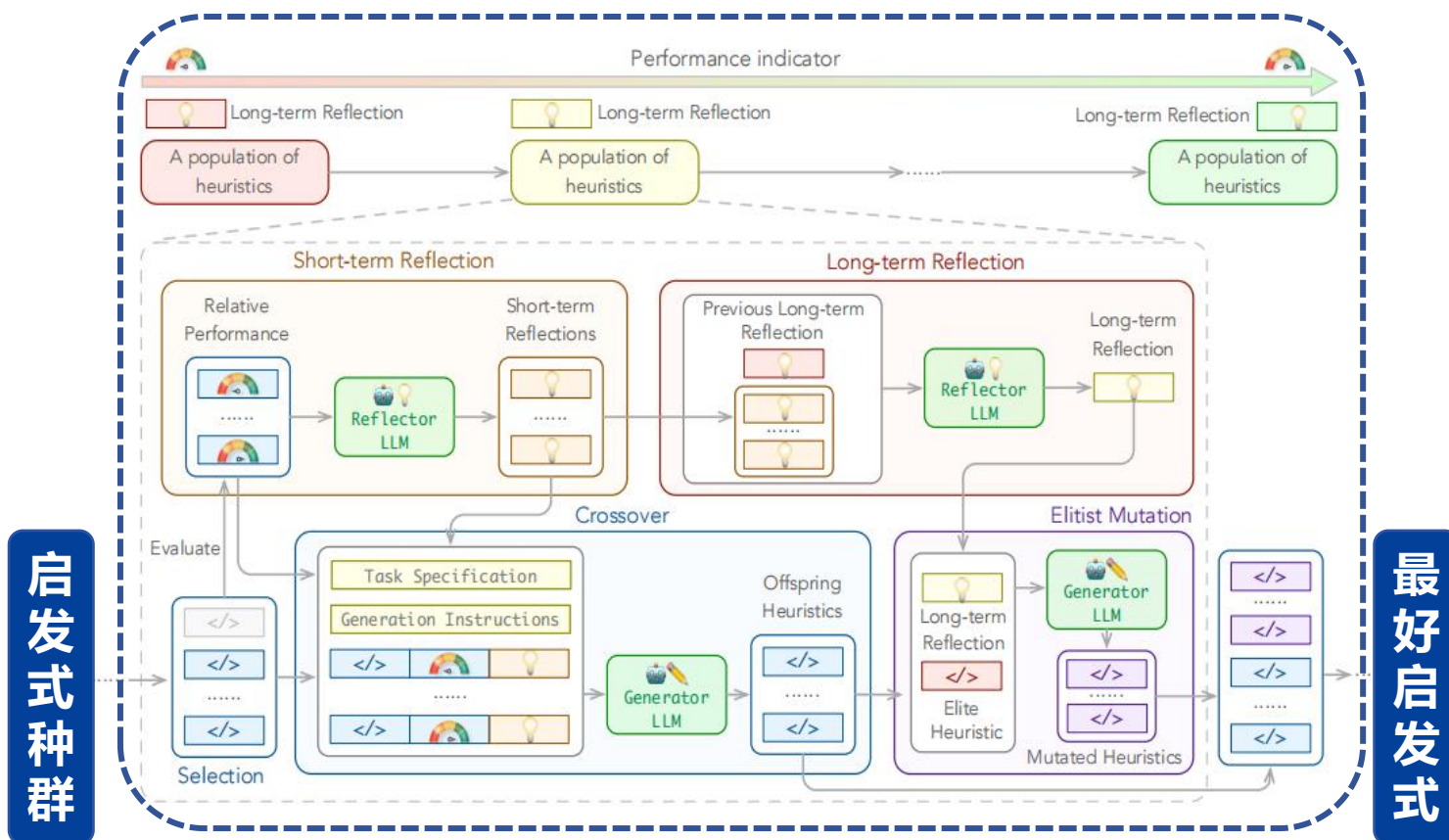
FunSearch

用户提供问题的 Specification，包括**评价函数、初始待优化函数以及可选的程序骨架**。系统将已有的正确程序及其得分保存在 Programs Database 中，并从数据库里优先选取性能较好的多个程序组合成 Prompt，交给预训练 LLM 生成一批新的候选程序。随后，Evaluator 会将这些程序真正嵌入求解流程并在多个训练实例上运行，无法执行、超时或产生非法结果的程序被淘汰，合法程序则根据求解质量获得分数并重新存入程序数据库。随着“选择已有程序—构造 Prompt — LLM 生成新代码—执行评价—更新数据库”这一闭环不断重复，表现优异的代码会更频繁地被选作后续生成的参考，而岛屿式种群机制还会维持程序多样性、避免搜索过早陷入局部最优。最终，FunSearch 从数据库中**取出一个实际评价得分最高的程序，作为自动发现的最佳启发式算法**。

研究背景 - [NeurIPS `24] ReEvo

ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution

最终得到一个性能最好的启发式算法



ReEvo

上面是系统每次迭代的结果，每一轮都会包括一个启发式种群和一个长期反思结果。下面是每一轮具体的实现细节，先从上一轮的启发式种群里面选择出来一对父代，然后送到上面生成短期反思，然后将短期反思和这对父类送到下面进行交叉生成子代启发式算法。短期反思还会送到长期反思里面，和历史的长期反思结合形成新的长期反思，最后根据这个新的长期反思以及产生的子代启发式算法进行突变，得到突变后的启发式算法，然后将突变后的启发式算法和交叉后的子代启发式算法结合为新的启发式种群。然后经过多轮迭代之后，ReEvo 在该启发式种群里面搜索得到一个实际性能最好的启发式算法。

研究背景

传统做法有哪些不足？

寻找**平均性能最好的单一启发式算法**的方法泛化性不好

不足：

- 1、在各种各样的问题实例上面**找性能最好的一个启发式算法很困难**
- 2、在**训练实例上的平均性能好**，但他在自己没有见过的问题实例上面**性能不一定好**

解决泛化性不足的常见办法：

算法组合

但是给全部的问题实例都设计一个启发式算法的做法不现实

提出了 EoH-S ， 本文的做法：

互补的自动启发式算法集合

- diversity-aware memetic search 模因搜索（由下面两个部分组成）
 - Complementary-aware Search (CS) 互补感知搜索
 - Local Search (LS) 局部搜索
- complementary population management 互补种群管理

提纲

1

研究背景

3

系统设计

4

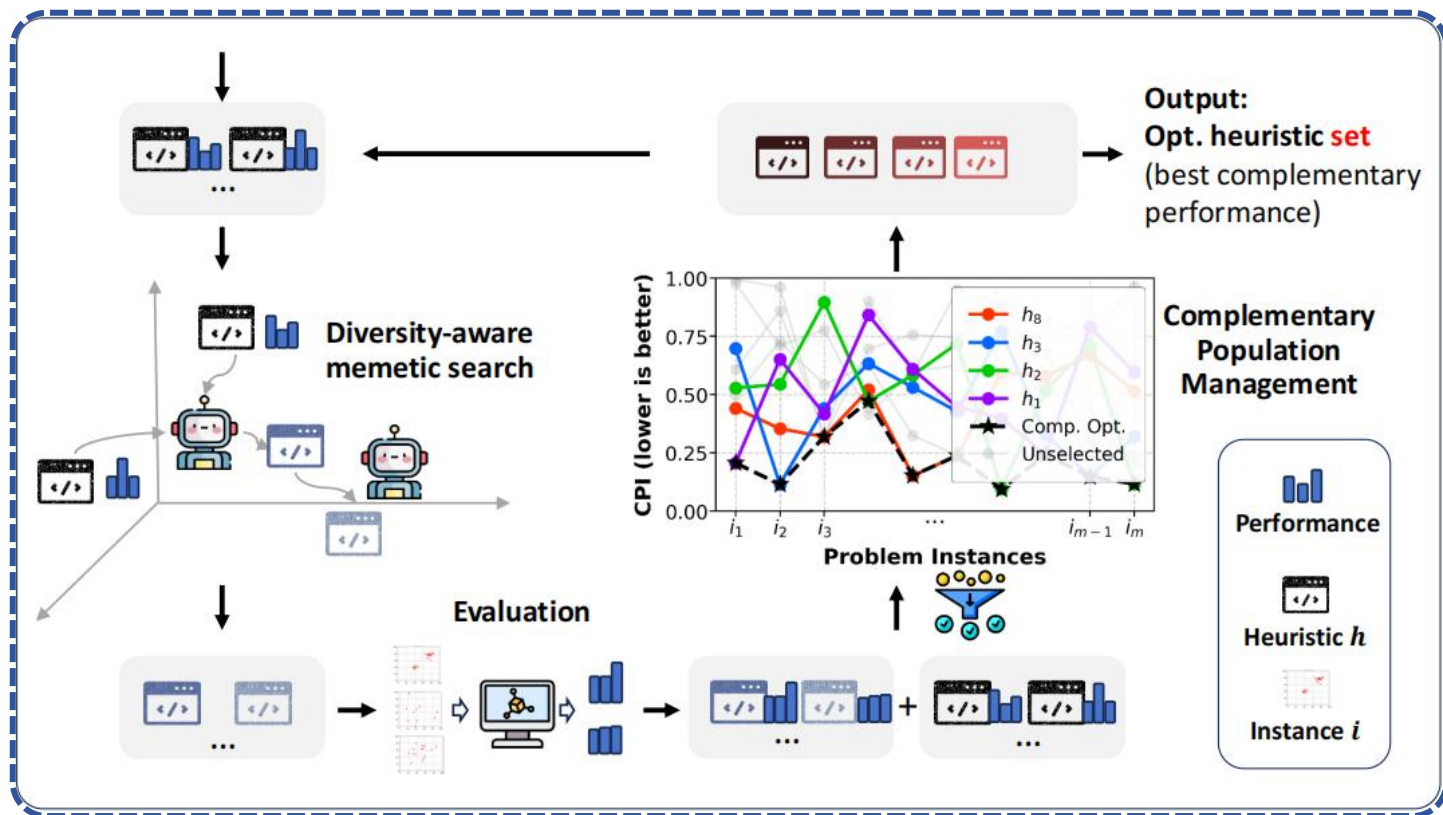
实验评估

5

小结

系统设计

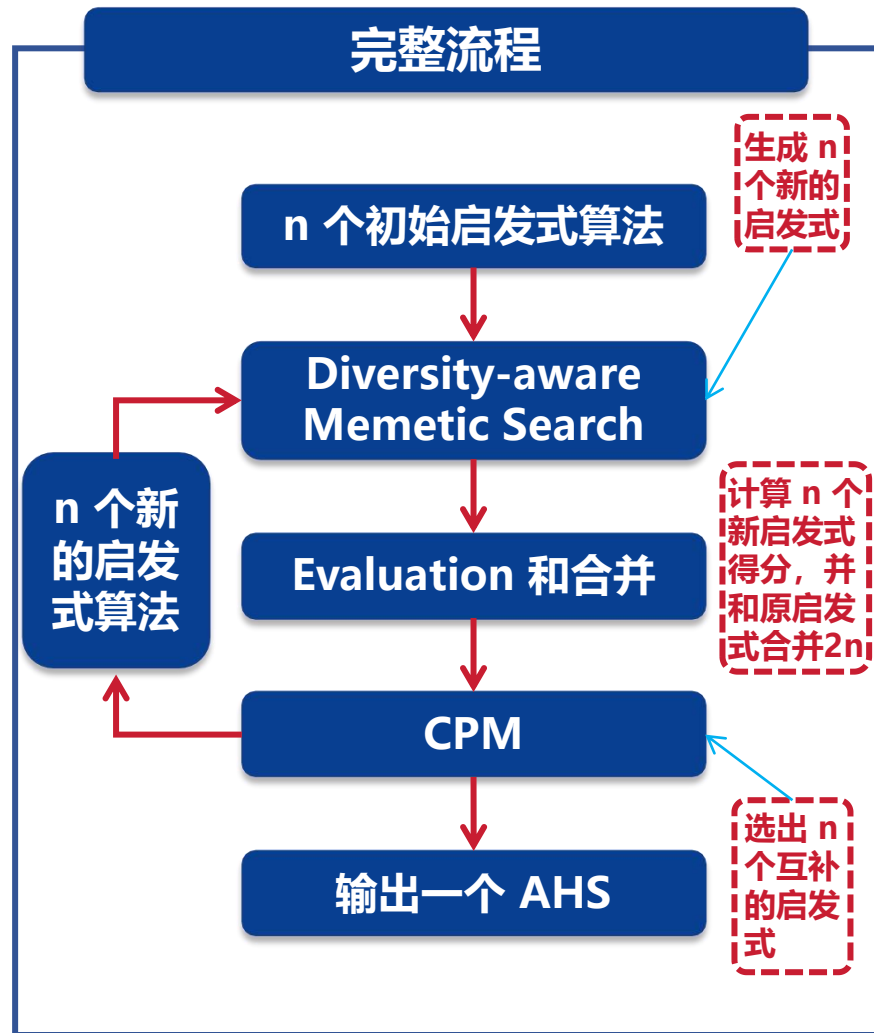
EoH-S 完整进化流程



AHS

Automatic Heuristic Set 自动启发式算法集合

完整流程

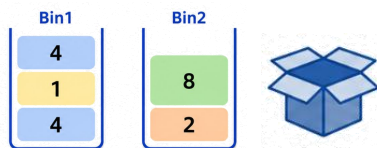


Automated Heuristic Set Design (AHSD) - 问题建模

任务和问题实例

T_{OBP} : 在线装箱问题

T_{OBP} : 每当一个新物品到达时, 应该把它放入哪个箱子, 目标是尽量减少使用的箱子数量



实例1

[6, 4, 5, 5]

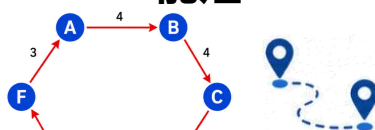
实例2

[8, 2, 2, 2, 6]

物品数量大小不同

T_{TSP} : 旅行商问题

T_{TSP} : 给定当前位置和还未访问的城市, 下一步应该访问哪个城市, 目标是使行驶距离尽可能短



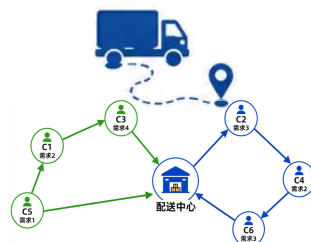
实例1

$\{(0,0), (0,1), (1,0), (1,1)\}$

城市坐标数量不同

T_{CVRP} : 容量车辆路径问题

T_{CVRP} : 车辆从仓库出发, 在汽车容量限制下, 每一步应该访问哪个客户, 目标是最小化所有配送路线的总距离



不同的仓库和客户坐标, 以及不同的客户需求和车容量

AHSD Problem

给定一个启发式算法设计任务 T
该任务有 m 个问题实例

$$I = \{i_1, \dots, i_m\}$$

AHSD 的目标:

自动的生成启发式算法集合 H , 在 m 个问题实例上面的互补性很好, 也就是互补性函数 $F(H)$ 是最小的, 即 CPI 是最小的

$$H = \{h_1, \dots, h_k\}$$

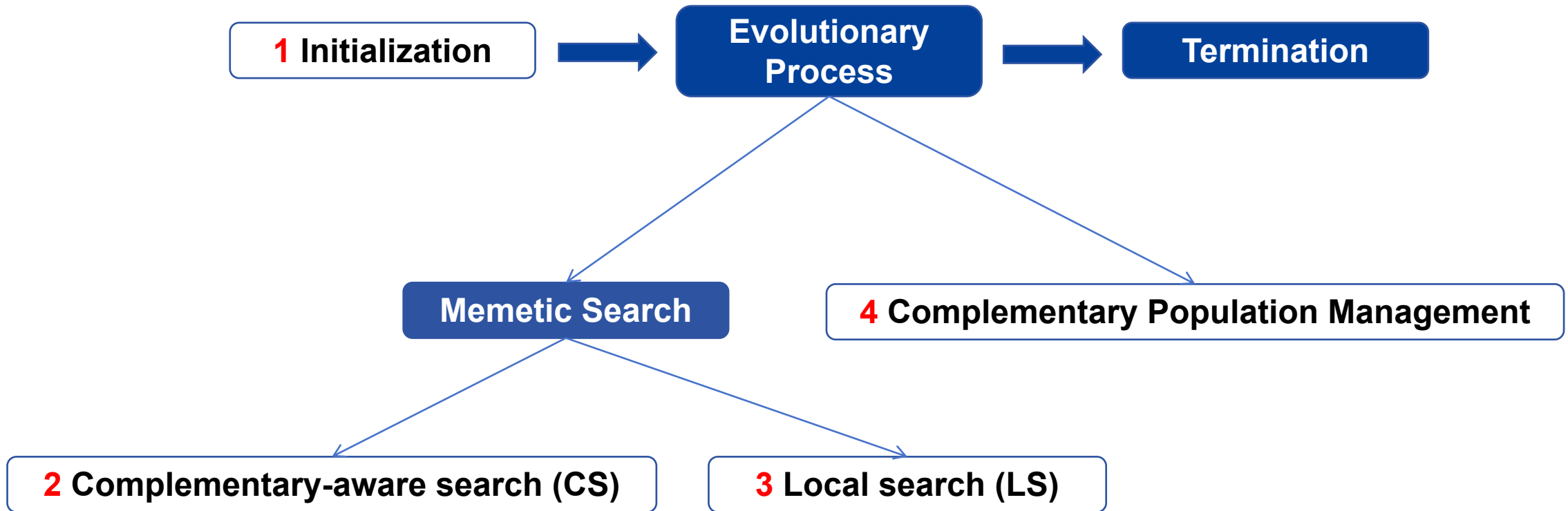
$F(H)$ 是 Complementary Performance Index, 也叫作 CPI (互补性能分数, 越小越好)

$$\mathcal{F}(H) = \frac{1}{m} \sum_{i \in I} f_{H,i}^*$$

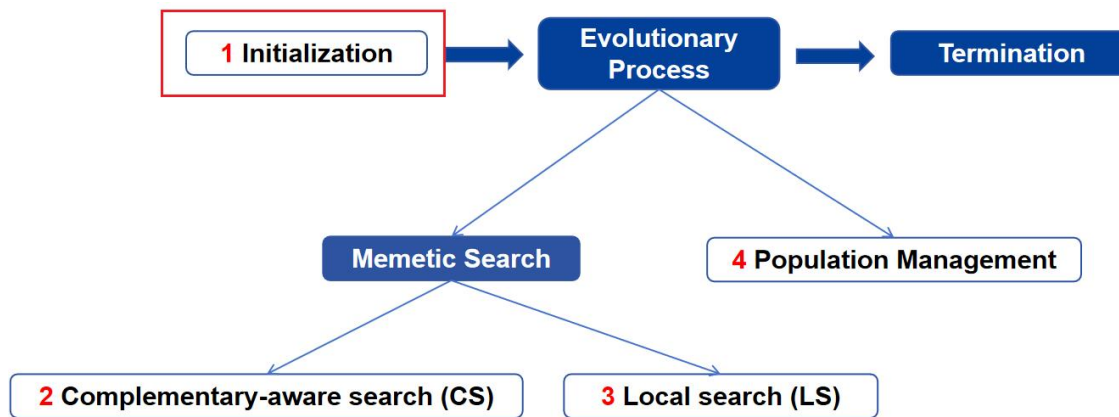
其中, $f_{H,i}^* = \min_{h \in H} f_i(h)$
他指的是, 启发式算法集合里面的每一个算法对于问题实例 i 的所有性能分数中最小的那一个 (性能得分越小越好)

系统设计

Evolution of Heuristic Set



1 Initialization



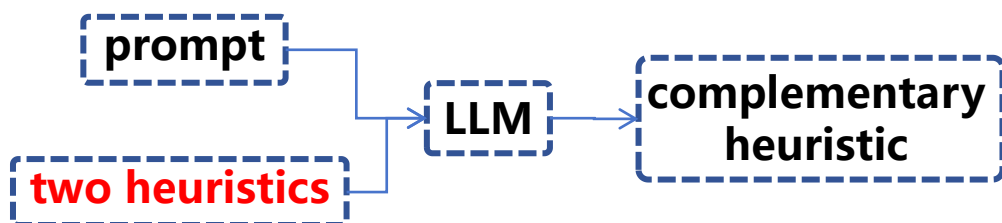
使用 LLM 来生成一个包含 n 个启发式算法的种群
该种群中可以包含一个或者多个人工设计的启发式算法

系统设计

Memetic Search

2 Complementary-aware search (CS)

目的：
深度探索各种各样的启发式算法，增加启发式算法集合的互补性能



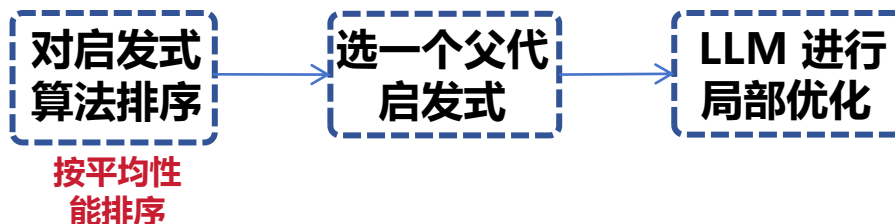
选取两个算法的规则：
用曼哈顿距离来表示，距离越大表示两个算法之间的互补性越好

$$d_{h_a, h_b} = \sum_{j=1}^m |f_{i_j}(h_a) - f_{i_j}(h_b)|$$

例子： h_a 和 h_b 分别在 4 个问题实例上面的分数向量
 $F(h_a)=[1, 8, 3, 9]$
 $F(h_b)=[8, 2, 7, 1]$
 $d_{h_a, h_b}=7+6+4+8=25$

3 Local search (LS)

目的：
进一步优化性能比较好的启发式算法



4 Population M

总结

只使用 CS，会不断生成差异很大的算法，但会缺少精细优化。

只使用 LS，会围绕少量优秀算法反复修改，容易陷入局部最优，算法之间也会越来越相似。

CS 负责探索新方向。

LS 负责改进已有方向。

4 Complementary Population Management (CPM)

m 个问题实例
2n 个候选启发式算法

$H_k = \{h_1, \dots, h_k\}$
当前的启发式集合

h_i 为候选启发式算法

$$\Delta CPI(h_i | H_k) = \sum_{j=1}^m \min(f_{i_j}(h_i) - f_{H_k, i_j}^*, 0)$$

越小越好

已知 H_k 的情况加入 h_i , 整个集合的互补性分数的变化

具体的步骤

Initialization

从 2n 个候选启发式集合里面选出平均性能最好的启发式作为 h_i

Iterative Selection

为剩下 $2n-k$ 个候选启发式计算 $\Delta CPI(h_i | H_k)$, 选择值最小的启发式加入集合中, 并更新 $f_{H_{k+1}, i_j}^* = \min(f_{H_k, i_j}^*, f_{i_j}(h_i))$, 即更新整个启发式集合对于问题实例 i_j 的性能分数

Terminate

当集合当中的启发式算法的数量为 n 时, 停止迭代

提纲

1

研究背景

3

系统设计

4

实验评估

5

小结

实验评估

任务和问题实例

任务和问题实例

T_{OBP} : 在线装箱问题

T_{OBP} : 每当一个新物品到达时, 应该把它放入哪个箱子, 目标是尽量减少使用的箱子数量

训练:

128 个问题实例

箱子容量 100

物品数量 200 ~ 2k

测试:

箱子容量 200、500

物品数量 1k、5k、10k

6种配置, 每种5个实例



T_{TSP} : 旅行商问题

T_{TSP} : 给定当前位置和还未访问的城市, 下一步应该访问哪个城市, 目标是使行驶距离尽可能短

训练:

128 个问题实例

城市数量 10~200

测试:

城市数量 50、100、200、500



T_{CVRP} : 容量车辆路径问题

T_{CVRP} : 车辆从仓库出发, 在容量限制下, 每一步应该访问哪个客户, 目标是最小化所有配送路线的总距离

训练:

256 个问题实例

节点数量 20~200

车辆容量 10~150

测试:

客户数量 50~500



训练是指使用这些实例来评价 LLM 生成的启发式, 并据此让 EoH-S 不断选择、修改和进化启发式

实验评估

实验设置

baseline:

- Random: 不进行迭代搜索, 反复让LLM独立生成启发式
- 1+1 ESP: 使用简单的一父一子进化策略
- FunSearch: 使用LLM和多岛式进化搜索程序
- EoH: 同时进化启发式思想和代码
- ReEvo: 加入短期和长期反思
- MEOH: 从多目标角度进化启发式
- MCTS-AHD: 使用蒙特卡洛树搜索探索启发式
- CALM: 协同进化算法和语言模型

比较方法:

- 1、直接比较(**Direct comparison**): ,
在 OBP 上, 直接使用原论文的最优启发式算法来比较
- 2、受控制的对比(**Controlled comparison**)
对 EoH / FunSearch / ReEvo 使用相同训练实例重新训练, 公平比较

统一的实验配置:

平台: LLM4AD

模型: DeepSeek-V3

最大评估次数: $N_{\max} = 2000$

种群规模: $n = 10$

硬件: Intel i7-9700 CPU

实验评估

在线装箱任务上面的实验

Methods	Training (c100)			Testing (c200-500)					
	n200-500	n500-1k	n1k-2k	n1k_c200	n1k_c500	n5k_c200	n5k_c500	n10k_c200	n10k_c500
First Fit	0.0652	0.0318	0.0387	0.0240	0.0124	0.0173	0.0075	0.0163	0.0055
Best Fit	0.0627	0.0310	0.0372	0.0220	0.0124	0.0161	0.0070	0.0154	0.0050
Random*	0.3638	0.2988	0.1489	0.1939	1.3508	0.0376	0.2816	0.0189	0.1402
1+1 EPS*	0.2482	0.2180	0.1246	0.0569	0.2390	0.0101	0.0439	0.0055	0.0224
FunSearch*	0.2510	0.2000	0.1326	0.0827	0.6169	0.0193	0.2260	0.0120	0.1211
EoH*	0.1216	0.1169	0.0750	0.0190	0.0124	0.0044	0.0050	0.0041	0.0042
MEoH*	0.0991	0.0749	0.0547	0.1037	0.0149	0.0417	0.0035	0.0326	0.0015
CALM*	0.0805	0.0300	0.0470	0.0120	0.0124	0.0048	0.0030	0.0034	0.0025
ReEvo*	0.0877	0.0344	0.0556	0.0120	0.0124	0.0074	0.0040	0.0068	0.0025
MCTS-AHD*	0.0974	0.0360	0.0596	0.0150	0.0124	0.0046	0.0040	0.0034	0.0022
FunSearch	0.0623	0.0298	0.0367	0.0213	0.0124	0.0159	0.0070	0.0152	0.0051
FunSearch Top10	0.0608	0.0289	0.0361	0.0200	0.0124	0.0153	0.0065	0.0123	0.0038
EoH	0.0619	0.0307	0.0371	0.0240	0.0133	0.0182	0.0075	0.0170	0.0059
EoH Top10	0.0618	0.0299	0.0368	0.0216	0.0124	0.0158	0.0065	0.0152	0.0051
ReEvo	0.0604	0.0298	0.0362	0.0217	0.0133	0.0162	0.0068	0.0155	0.0053
ReEvo Top10	0.0580	0.0285	0.0349	0.0203	0.0124	0.0125	0.0063	0.0108	0.0049
EoH-S	0.0515	0.0230	0.0314	0.0113	0.0124	0.0033	0.0025	0.0014	0.0010

比较指标：
算法所使用的箱子
数量相对于理论下
界的平均相对差距，
越小越好

$$\text{Gap}_i(h) = \frac{B_i(h) - LB_i}{LB_i}$$

n 表示物品数量
c 表示箱子容量

实验目的

- 1、EoH-S 设计的“互补启发式集合”是否比现有方法设计的单个启发式更好
- 2、EoH-S 的优势是否只是因为它用了 10 个启发式
- 3、检验生成的启发式能否泛化到更大规模、不同容量的未见实例

结论

EoH-S 方法优于或者等于之前的所有方法

实验评估

旅行商问题（TSP）和容量车辆路径问题（CVRP）上面的实验

TSP					
Methods	Training	Testing			
		50	100	200	500
FunSearch	0.156	0.144	0.149	0.169	0.185
FunSearch Top10	0.124	0.091	0.105	0.124	0.157
EoH	0.152	0.142	0.154	0.168	0.194
EoH Top10	0.127	0.079	0.123	0.134	0.175
ReEvo	0.155	0.167	0.183	0.223	0.221
ReEvo Top10	0.117	0.092	0.123	0.157	0.170
EoH-S	0.097	0.040	0.065	0.090	0.111

CVRP					
Methods	Training	Testing			
		50	100	200	500
FunSearch	0.294	0.315	0.365	0.296	0.241
FunSearch Top10	0.245	0.267	0.309	0.247	0.214
EoH	0.276	0.274	0.299	0.261	0.221
EoH Top10	0.230	0.172	0.232	0.218	0.198
ReEvo	0.265	0.274	0.296	0.256	0.203
ReEvo Top10	0.240	0.214	0.249	0.173	0.178
EoH-S	0.173	0.135	0.188	0.180	0.169

实验目的

- 1、EoH-S 设计的“互补启发式集合”是否比现有方法设计的单个启发式更好
- 2、EoH-S 的优势是否只是因为它用了 10 个启发式
- 3、检验生成的启发式能否泛化到更大规模、不同容量的未见实例

结论

EoH-S 方法优于或者等于之前的所有方法

使用公开、标准的组合优化数据集进行验证

BPPLib: 在线装箱问题基准库
TSPLib: 旅行商问题基准库
CVRPLib: 容量约束车辆路径问题基准库

Benchmarks	EoH		ReEvo		EoH-S
	Top 1	Top 10	Top 1	Top 10	
BPPLib Sch_1	0.153	0.153	0.153	0.153	0.152
BPPLib Sch_2	0.142	0.142	0.142	0.142	0.141
BPPLib IRUP	0.077	0.075	0.079	0.074	0.072
BPPLib NonIRUP	0.077	0.076	0.080	0.075	0.073
BPPLib Scholl_H	0.138	0.138	0.138	0.138	0.095
TSPLib	0.184	0.173	0.220	0.165	0.093
CVRPLib A	0.326	0.277	0.329	0.261	0.231
CVRPLib B	0.374	0.310	0.352	0.250	0.183
CVRPLib E	0.334	0.268	0.305	0.257	0.242
CVRPLib F	0.539	0.493	0.687	0.547	0.427
CVRPLib M	0.461	0.377	0.442	0.372	0.299
CVRPLib P	0.273	0.206	0.270	0.188	0.169
CVRPLib X	0.270	0.237	0.270	0.224	0.196

最多降低
了 31.2%

降低了
43.6%

最多降低
了 26.8%

最少降低
了 5.8%

实验目的

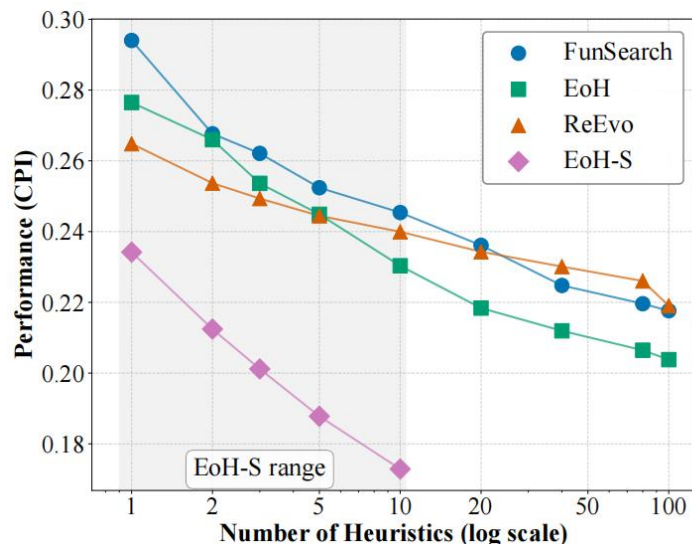
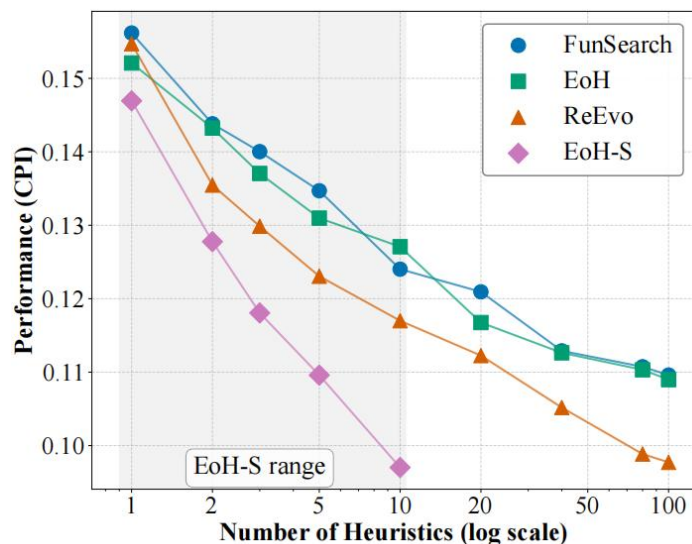
- 检验 EoH-S 在标准公开基准数据集上的泛化能力和鲁棒性
- 1、面对不同规模和分布的真实基准实例时，是否仍然有效
 - 2、EoH-S 的 10 个互补启发式是否比 EoH、ReEvo 的单个最佳启发式以及简单选出的 Top10 启发式集合更好

结论

EoH-S 方法优于之前的方法

实验评估

EoH-S 启发式集合互补性能的实验



EoH-S 选择大小为 10 的启发式集合
其他方法从候选启发式里面选出性能最好的 100 个
不断增加启发式的数量，比较 CPI

实验结果

- 1、EoH-S 设计的启发式集合始终优于其他对比方法设计的启发式集合
- 2、即使其他方法使用 100 个启发式，也难以达到 EoH-S 仅使用 10 个启发式所取得性能
- 3、EoH-S 集合中的每个启发式都对整体性能作出了有效贡献
EoH-S 的性能曲线下降更加陡峭

结论

EoH-S 方法启发式集合里面的启发式算法具有更好的互补性

实验评估

消融实验

OBP

No.	Settings	Gap
1	First Fit	0.0413
2	Best Fit	0.0399
3	EoH-S w/o LS	0.0336
4	EoH-S w/o CS	0.0335
5	EoH-S w/o CPM	0.0373
6	EoH-S n5	0.0333
7	EoH-S n15	0.0339
8	EoH-S n20	0.0337
9	EoH-S n10	0.0326

实验结果

1、由 3 4 5 可以看出来 LS、CS、CPM 都对性能提升有用，其中 **CPM 提升最多**
2、6 7 8 9 四种规模的结果都比较接近，并且均优于 First Fit 和 Best Fit，说明 EoH-S 对种群规模具有一定的鲁棒性

结论

EoH-S 方法里面的 LS、CS、CPM 都对性能提升起作用，其中 CPM 最关键

提纲

1

研究背景

3

系统设计

4

实验评估

5

小结

小结

本文总结：

传统的 LLM 自动启发式设计方法，通常使用 LLM 搜索一系列候选程序，最终在训练实例上面选择平均性能最好的一个启发式算法。但这种方法存在两个问题：

一是在多个问题实例上面找一个性能最好的启发式算法很困难。

二是在训练实例上的平均性能好，但在自己没有见过的问题实例上面性能不一定好。

为此，本文提出自动启发式集合设计 AHSD，并设计 EoH-S，由 LLM 生成一个规模小且互补的启发式集合。其中，**模因搜索 (Memetic search)** 通过互补感知搜索 (CS) 探索不同的启发式，并通过局部搜索 (LS) 提高单个启发式的质量，**互补种群管理 (CPM)** 则利用每个算法在不同实例上的性能向量，保留能够弥补当前集合弱点的成员。

实验表明，EoH-S 在 OBP、TSP 和 CVRP 等任务上整体优于现有方法，同时表现出更强的跨规模和跨分布泛化能力。

小结

别人的好方法能不能用到我们的场景？

我们的场景是计算调度的场景。可以让 LLM 根据当前系统的负载信息生成一个互补的调度函数集合，根据不同的系统负载情况以及不同的任务类型选择不同的启发式算法。

任务 T 是设计一个任务调度策略，决定“当前 GPU 下一次运行哪个任务”或“任务分配到哪个 CPU / GPU 上面来运行”

问题实例 i 就是具体的系统状态，包括任务数量、到达时间、任务长度、资源需求和当前负载

启发式算法 h 就是计算调度的函数

能不能进一步提高？

EoH-S 用 CPI 评价集合时，是事后知道每个问题最适合使用哪个启发式算法。他没有涉及到已知一个启发式算法集合，在该集合当中为每一个任务选择哪个启发式算法的效果更好。但在真实调度中，新任务到达时并不知道应该选哪一个策略。

可以设置一个轻量级选择器，根据系统当前状态，快速地选择最合适的调度启发式算法。

小结

问题可以泛化吗？

在同一个调度任务中可能出现很多种不同的情况，例如低负载、高并发和突发负载、短任务、长任务和混合任务，某个调度策略可能适合短任务，另一个适合高负载，还有一个适合节能，很难由一个固定策略覆盖全部情况。

因此，可以把一次具体的工作负载和系统配置看作一个“问题实例”，利用 LLM 来生成一个互补调度策略集合。

Thank you!